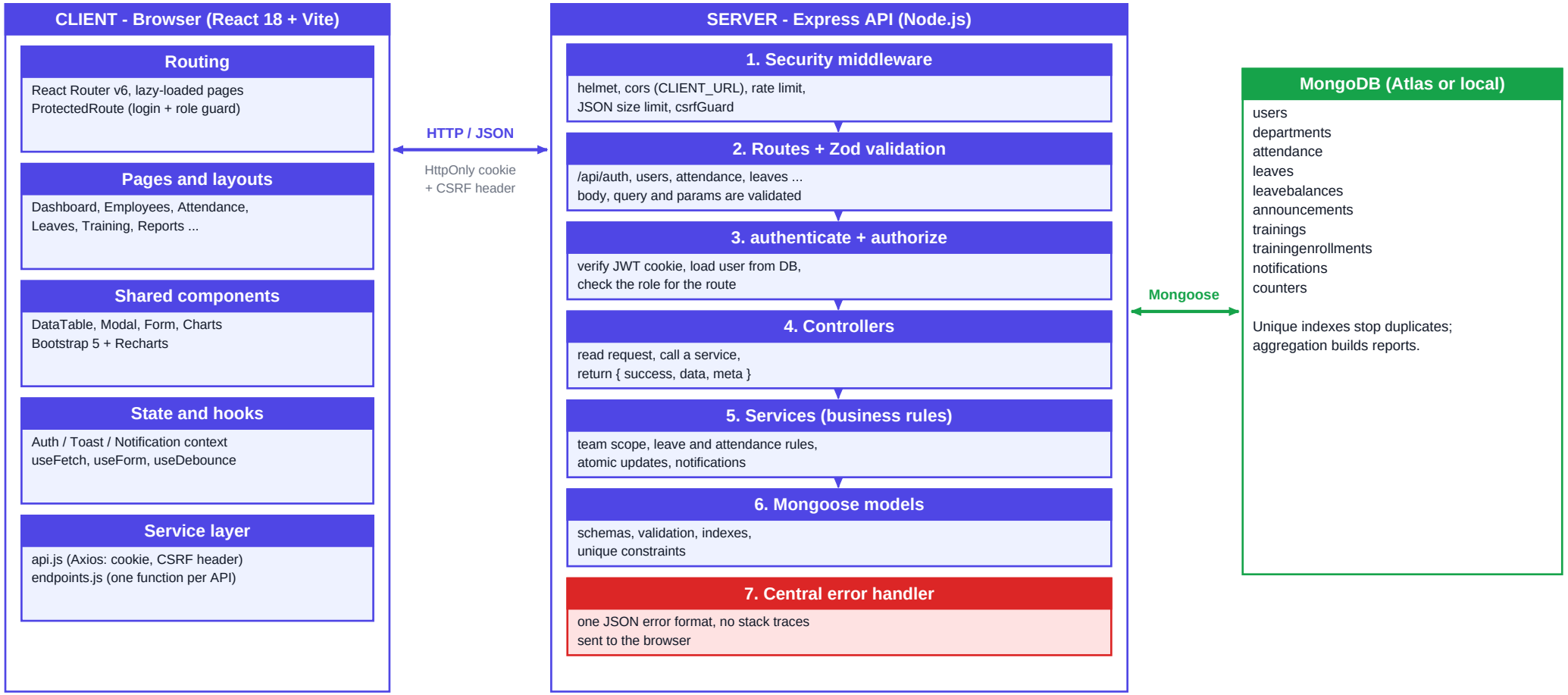
Technical Architecture

How the StaffSync browser app, Express API and MongoDB database fit together



Configuration (.env, never committed)	Authentication	Not included (kept simple)
MONGODB_URI, JWT_SECRET, CLIENT_URL, SETUP_KEY, ORG_TIMEZONE, WORK_START, WEEKEND_DAYS, HOLIDAYS	JWT (8 h) in an HttpOnly cookie, bcrypt passwords, sessions end on password change or deactivation	No email/SMS, WebSockets, microservices or queues. Notifications are in-app and polled every 60 s.

Technology stack and version decisions

Version ranges come from package.json; npm installs the newest compatible release of each.

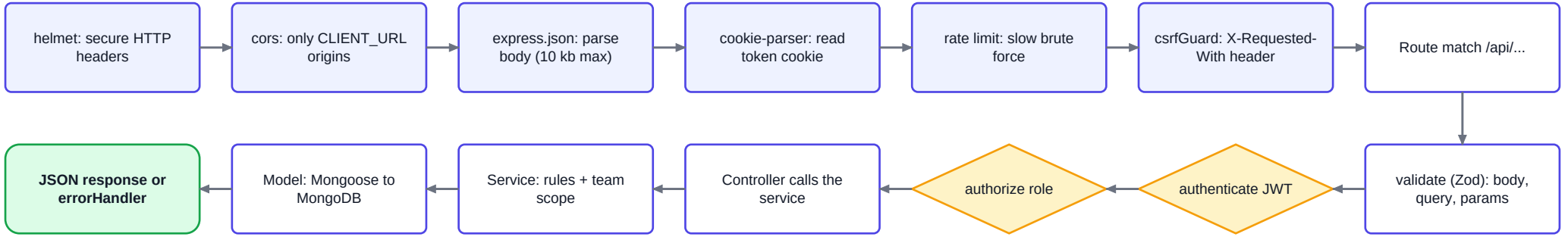
Layer	Technology	Version range	Why it is used
Runtime	Node.js	20 or newer (LTS)	Runs JavaScript on the server; same language as the front-end.
Web framework	Express	^4.21	Mature, simple middleware model that is easy to explain and debug.
Database	MongoDB + Mongoose	Mongoose ^8.9	Flexible documents, schemas with validation, indexes and aggregation pipelines.
Authentication	jsonwebtoken, bcryptjs, cookie-parser	^9.0 / ^2.4 / ^1.4	Signed session tokens in HttpOnly cookies; slow salted password hashing (pure JS, so no native build problems on Windows).
Validation	Zod	^3.24	Describes the allowed shape of every request; drops unknown fields (mass-assignment protection).
Security	helmet, cors, express-rate-limit	^8.0 / ^2.8 / ^7.5	Safe HTTP headers, an origin allow-list and brute-force limits.
Logging / config	morgan, dotenv	^1.10 / ^16.4	Request logs in development; environment variables kept out of the code.
Front-end core	React + Vite	React ^18.3, Vite ^6	Component UI with hooks; very fast dev server and builds.
Front-end libraries	react-router-dom, axios, bootstrap, recharts, react-icons	^6.28 / ^1.7 / ^5.3 / ^2.15 / ^5.4	Routing, HTTP client, responsive styling, charts, icons.
Testing	Vitest, Supertest, mongodb-memory-server, React Testing Library	^2.1 / ^7 / ^10 / ^16	API integration tests on an in-memory database and component tests.

Important version decisions

- **Express 4, not 5:** the most widely documented version; nothing in this project needs Express 5 features.
- **Zod 3, not 4:** stable API that the code relies on (error issues list).
- **React 18:** matches Recharts and React Testing Library versions used here.
- **Vite proxy in development:** the browser calls /api on port 5173 and Vite forwards it to port 5000, so there are no CORS or cookie problems and no client .env file is needed.
- **bcrypt cost 12** in production (4 in automated tests so the suite stays fast).

Request pipeline (backend)

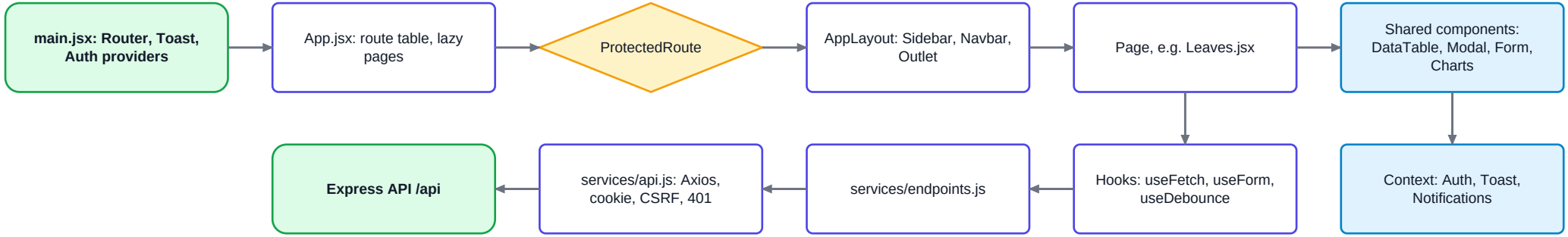
Every API call passes through the same ordered chain. Any step can stop the request with an error response.



Step	Component	What it does	Failure result
1-3	helmet, cors, express.json	Adds security headers, allows only configured browser origins, parses JSON (rejects bodies over 10 kb).	413 too large / 400 malformed JSON
4-5	cookie-parser, rate limiter	Makes req.cookies available; limits login-type routes to 20 attempts per 15 minutes per IP.	429 too many requests
6	csrfGuard	POST, PUT, PATCH and DELETE must carry X-Requested-With: StaffSync (only our own JavaScript adds it).	403
8	validate (Zod)	Parses body, query and params into clean typed data and removes unknown fields.	400 with a field error list
9	authenticate	Verifies the JWT cookie, reloads the user, rejects deactivated users and tokens older than the last password change.	401
10	authorize	Checks the user role against the roles allowed for the route.	403
11-13	controller, service, model	Controller calls a service; the service applies business rules and scope (HR all, manager team, employee self) and uses models.	400 / 403 / 404 / 409 (AppError)
14	errorHandler	Maps validation, cast, duplicate-key and custom errors to one JSON format; hides internals in production.	Consistent { success:false, message }

Front-end architecture

How a screen is built and how it talks to the API



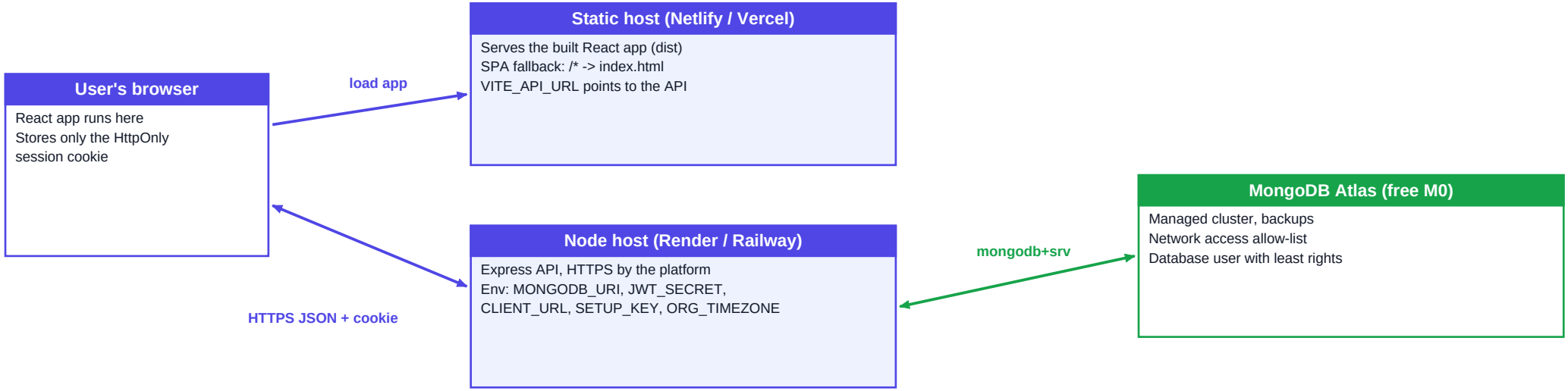
- **Pages** are route-level screens loaded on demand with React.lazy, so the first download stays small.
- **Pages never call Axios directly.** They call functions in services/endpoints.js; api.js adds the cookie, the CSRF header and logs the user out on any 401.
- **useFetch** handles loading, error and out-of-order responses; **useForm** keeps form values and server field errors.
- **Context** is used only for things many screens need (current user, toasts, unread count). Redux is not needed.
- **ProtectedRoute** is a user-experience guard. The real access control is on the server.

Backend folders	Frontend folders

`server/src/`
`  config/  env.js, db.js`
`  models/  User, Department, Attendance, Leave,`
`  LeaveBalance, Announcement, Training,`
`  TrainingEnrollment, Notification, Counter`
`  routes/  URL + middleware order`
`  controllers/  HTTP in and out`
`  services/  business rules`
`  middleware/  auth, csrf, validate, errorHandler`
`  validators/  Zod schemas`
`  utils/  dates, response, AppError`
`  seed/  fictional demo data`
`  app.js  server.js`
`server/tests/  Vitest + Supertest`
	`client/src/`
	`  components/  Sidebar, Navbar, DataTable, Modal ...`
	`  layouts/  AppLayout`
	`  pages/  Dashboard, Employees, Attendance,`
	`  Leaves, Trainings, Reports ...`
	`  features/  dashboard widgets per role`
	`  services/  api.js, endpoints.js`
	`  hooks/  useFetch, useForm, useDebounce`
	`  context/  Auth, Toast, Notification`
	`  routes/  ProtectedRoute`
	`  utils/  format, errors, navigation`
	`  __tests__/  React Testing Library`
	`  App.jsx  main.jsx`

Deployment and security architecture

Planned layout. Nothing has been deployed yet; see docs/DEPLOYMENT.md for the steps.



- **Same-site layout (recommended):** serve the app and the API under one domain via a proxy; keep CROSS_SITE_COOKIES=false.
- **Two-domain layout:** set CROSS_SITE_COOKIES=true (SameSite=None; Secure) and use HTTPS on both. Some browsers block third-party cookies, so test carefully.
- **Development:** localhost:5173 (Vite, proxies /api) to localhost:5000 (Express) to a local MongoDB or Atlas.

Security layers

Layer	Control
Transport	HTTPS in production; Secure cookies; Helmet headers.
Identity	bcrypt password hashes; JWT in HttpOnly cookie; invalidated after password change or deactivation; one-time hashed invite tokens.
Request	CORS allow-list; CSRF custom header; rate limiting; body size limit; Zod validation on body, query and params.
Access control	Role middleware per route plus scope rules in services (manager = own team, employee = self). Last HR admin cannot be removed.
Data	Unique indexes for duplicates; atomic conditional updates for leave approval, check-out and training seats; no salary or ID numbers stored.
Errors and secrets	No stack traces to clients in production; secrets only in .env (git-ignored); startup fails if JWT_SECRET is weak.